

CS151 Intro to Data Structures

Checkstyle

Stacks

JUnit

Warmup

You're given a Playlist interface with two methods: `nowPlaying()`, which returns the current song, and `skip()`, which moves on to the next song.

Write a `LinkedListPlaylist` class that implements `Playlist` as a `LinkedList`. `skip()` should throw an exception if the playlist is empty.

Add a recursive method `int remaining()` that returns how many songs are left.

```
public interface Playlist {  
    String nowPlaying(); // returns the current song  
    void skip();         // moves on to the next song  
}
```

Announcements

- Quiz today in Lab
- Lab3 due tonight
- HW2 due Friday Oct 2nd

Outline

- Checkstyle
- Stacks
- JUnit

Checkstyle

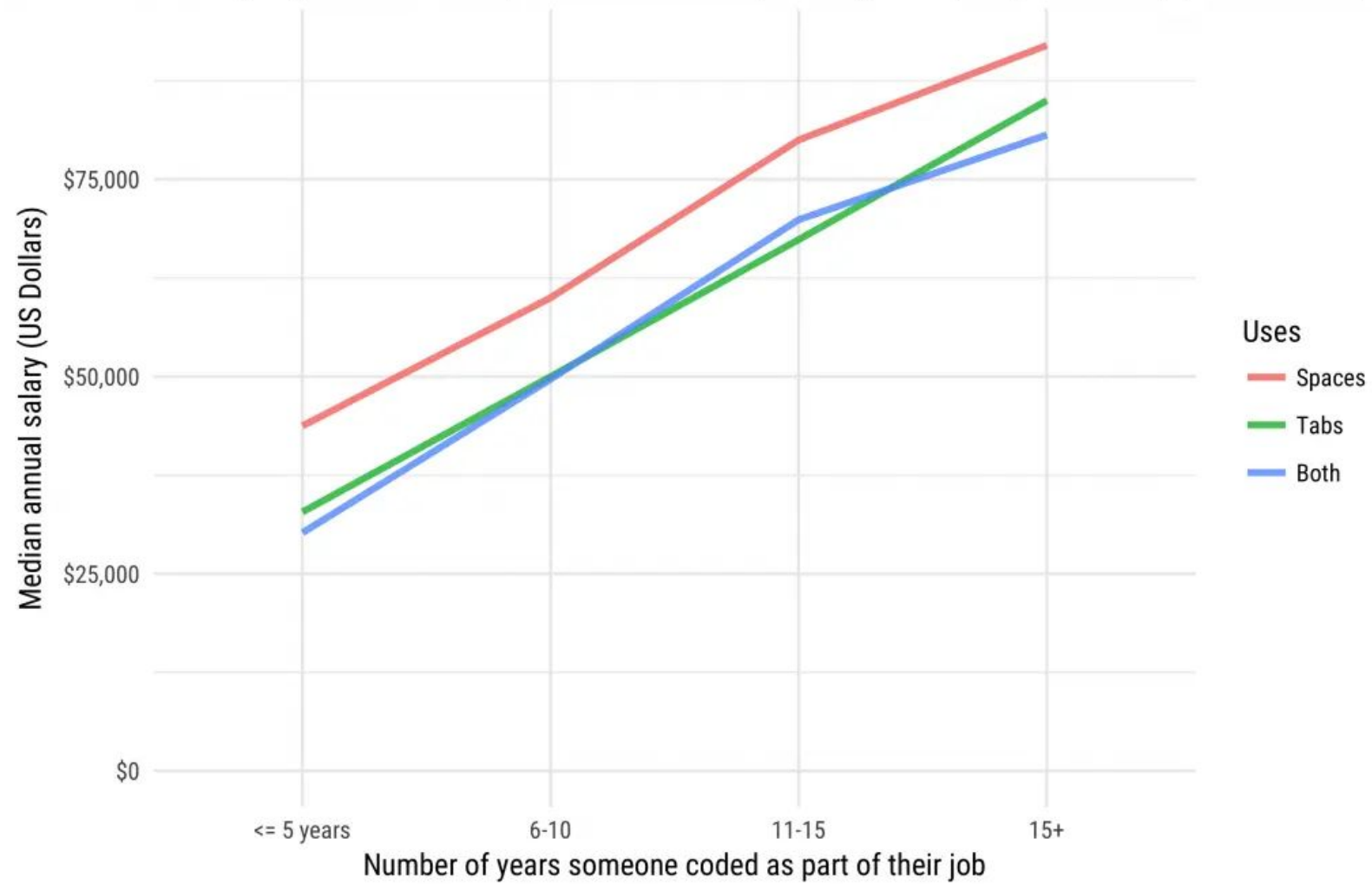
- A tool that ensures that the code adheres to a specified coding standard
- Helps with readability
 - Especially coding in teams
- Checkstyle is a **linter** static code analysis tool used to find and flag stylistic issues, and other issues in source code

CS151 Style Requirements

- Each file < 2000 lines
- No tabs
 - Tabs can be a different size on different computers and printers.
 - With spaces, the code will look the same regardless of the computer.
 - Our vim setup will automatically replace tabs with spaces
- Javadocs required
 - classes
 - methods
 - public and protected instance variables
 - must be properly formatted
- global const / final variable names must be all caps
- local variables and methods must be **camelcase**
- No star imports (import java.util.*)
- No unused imports
- Lines must be under 80 characters
- Methods must be under 150 lines
- and more :)

Salary differences between developers who use tabs and spaces

From 12,426 professional developers in the 2017 Developer Survey results, who provided tabs/spaces and salary



Javadoc review

Proper format for a class

- `/** .... */`
- First sentence must end in a period

```
/**
```

```
*  Description of what this segment of code does.
```

```
*/
```

Javadoc review

Proper format for a method

```
/** A method to return 0.  
* @return an integer 0  
* @param foo an input  
**/  
  
public int retZero(int foo) {  
    return 0;  
}
```

First sentence ends with a period

If there is a return value, must add a **@return** flag.

If there are any parameters, they must be marked with a **@param** flag followed by the param name

Checkstyle Example

```
java -jar checkstyle-8.16-all.jar -c cs151_checks.xml LookupZip.java
```

Checkstyle Requirements

HW03 onwards, all code must comply with the checkstyle

Future courses require it

Employers often require it

Makes working collaboratively much easier

Stacks

Stacks - FILO

- First In Last Out
- *stack* of plates in the dining hall
- Operations:
 - push
 - pop
 - peek
 - isEmpty

Stack Interface

```
public interface Stack<E> {  
    int size();  
    boolean isEmpty();  
    E pop();  
    E peek(); //does not modify the stack  
    void push(E element); //pushes to top of stack  
}
```

Stack Example - Browser History

- Coding using the Java Stack data structure (we won't implement our own yet)
- What happens if you call pop on an empty stack?
 - This is called a precondition.

Stack Example

Method	Return Value	Stack Contents
push(5)		

Stack Example

Method	Return Value	Stack Contents
push(5)	—	(5)
push(3)	—	(5, 3)
size()	2	(5, 3)
pop()	3	(5)
isEmpty()	false	(5)
pop()	5	()
isEmpty()	true	()
pop()	null	()
push(7)	—	(7)
push(9)	—	(7, 9)
top()	9	(7, 9)
push(4)	—	(7, 9, 4)
size()	3	(7, 9, 4)
pop()	4	(7, 9)
push(6)	—	(7, 9, 6)
push(8)	—	(7, 9, 6, 8)
pop()	8	(7, 9, 6)

Another Stack Example - Calculator

Postfix notation: A way to represent mathematical operations where *operators* come after their *operands*

- 3 4 +

Infix notation (normal): *operators* between *operands*

- 3 + 4

Postfix notation eliminates the need for parenthesis

Postfix Notation - Examples

What is the postfix notation of $(3 * 7)$?

- $3\ 7\ *$
- How would we use a stack for this? Let's draw it out
- Now, let's code it

Postfix Notation - Examples

What is the postfix notation of $(3 * 4 * 5)$

- $3\ 4\ 5\ *\ *$
- Draw out the stack!

What is the postfix notation of $(3 + 4 * 5)$. Think PEMDAS! Which operation should occur first. We read left to right

- $3\ 4\ 5\ *\ +$
- Draw out the stack!

Call Stacks

- Manages the flow of control in a program
- Data structure (stack) that stores the information about the current method
- As functions call and return, the call stack grows and shrinks

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

main
val = <f(15)>

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

f
Return val to main line 510
a = 15
y = 42
z = 13
x = <g(28)>

main
val = <f(15)>

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

g
Return val to f line 400
a=28
val = <h(56)>

f
Return val to main line 510
a = 15
y = 42
z = 13
x = <g(28)>

main
val = <f(15)>

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

h

Return val to g line 501
x=56
k=56
val = 57

g

Return val to f line 400
a=28
val = <h(56)>

f

Return val to main line 510
a = 15
y = 42
z = 13
x = <g(28)>

main

val = <f(15)>

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

g
Return val to f line 400
a=28
val = 57

f
Return val to main line 510
a = 15
y = 42
z = 13
x = <g(28)>

main
val = <f(15)>

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

f
Return val to main line 510
a = 15
y = 42
z = 13
x = 57

main
val = <f(15)>

The call stack

```
public class Lec8 {  
  
    public static int f(int a) {  
        int y = 42;  
        int z = 13;  
  
        int x = g(13 + a);  
  
        return x + y + z;  
    }  
  
    public static int g(double a) {  
        return h(a*2);  
    }  
  
    public static int h(double x) {  
        int k = (int) x;  
        return k+1;  
    }  
  
    public static void main(String[] args) {  
        int val = f(15);  
        System.out.println(val);  
    }  
}
```

main

val = 112

Call Stack

Keeps track of the local variables and return location for the current function

Allows program to jump from function to function without losing track of where the program should resume

stack frame: records the information for each function call:

- local variables
- address of where to resume processing after this function is complete.

computer only needs to ***add or remove items from the very top of the stack,***

stacks are a very useful data structure for ***FIRST IN LAST OUT*** processing

Stacks Summary

Simple and surprisingly useful data structure

First In Last Out (FILO)

Can store any number of items

User can only interact with the top of the stack:

- Push: add a new element to the top
- Pop: take off the top element
- Peek: view the top element without removing it

JUnit

JUnit

A *unit* testing framework for Java.

A “*unit*” is typically a method that we want to test in isolation

Let's write JUnit tests for our BrowserHistory

Using JUnit

Import Test Annotation Framework

```
import org.junit.Test;
```

- Write tests using @Test annotation

```
@Test
public void testEmpty() {
    ArrayStack<String> stack = new ArrayStack<String>(10);
    assertTrue(stack.isEmpty());
}
```

Testing Guidelines

Test every method for correct outputs:

- Try simple and complex examples

Every exception and error condition should be tested too

Write test cases first, then implement

- Will make it easy to know when you are done

Running Junit tests - run_junit.sh

```
javac -cp .:junit-4.13.2.jar $1.java
```

```
java -cp junit-4.13.2.jar:hamcrest-core-1.3.jar:. org.junit.runner.JUnitCore $1
```

Back to stacks...

Implementing a Stack with an Array

Goal: $O(1)$ operations

Array Stack Performance

Space complexity is

- $O(n)$

Runtime Complexity:

- push:
 - $O(1)$
 - what if we had an expandable array? $O(n)$
- Pop:
 - $O(1)$
- Peek:
 - $O(1)$

Summary

- Stacks are FILO data structures
 - can push or pop but can't retrieve non-top values
 - $O(1)$ operations
- Checkstyle is a tool to check if your code conforms to readability standards
- JUnit is a testing library
 - testing is important to verify your code's adherence to the specification
- HW2 and Lab3 due tonight!